PYTHON FOR DATA · 40 MIN

Data Loading and Storage

Data Preprocessing and Visualization · Unit 4

CSV files
Plain text tables
`read_csv`

JSON and Excel
Common data files
`read_json`

Databases
SQL tables
`read_sql`

Saving
Write results out
`to_csv`

~3 min

Four Jobs of Data Input and Output

Getting data into Python and back out.

1

Read

Load a file into a DataFrame.

`read_csv`

2

Adjust

Set headers, index and missing values.

`header names`

3

Write

Save a DataFrame to a file.

`to_csv`

4

Store

Keep data in Excel, pickle or a database.

`to_sql`

Almost everything is done with pandas, using read_ and to_ functions.

~4 min

Reading a CSV File

One line loads a whole file.

```
import pandas as pd

df = pd.read_csv("people.csv")
print(df)
print(df.shape)
```

Output

```
    name  age   city
0   Asha   21   Pune
1   Ravi   25  Delhi
2  Meena   23    Goa
(3, 3)
```

The file

people.csv holds a header line, then one line per person, split by commas.

The result

read_csv returns a DataFrame. shape gives (rows, columns).

~3 min

Read and Write Functions

Each file type has a matching pair.

File type	Read	Write
CSV or text	`read_csv`	`to_csv`
JSON	`read_json`	`to_json`
Excel	`read_excel`	`to_excel`
HTML table	`read_html`	`to_html`
Pickle	`read_pickle`	`to_pickle`
SQL database	`read_sql`	`to_sql`

Reading functions start with read_. Writing functions start with to_.

~4 min

Headers and Index

Use files that have no header row.

```
df = pd.read_csv("raw.csv",
                 header=None,
                 names=["id", "name", "age"],
                 index_col="id")
print(df)
print(df.index.tolist())
```

Output

```
    name  age
id
1   Asha   21
2   Ravi   25
[1, 2]
```

No header line

header=None says the first line is data. names= gives your own column names.

Row labels

index_col= makes one column the index, so the rows are labelled by id.

~3 min

Missing Values

Tell pandas which symbols mean missing.

```
df = pd.read_csv("scores.csv",
                 na_values=["-"])
print(df)
print(df["b"].isna().sum())
```

Output

```
   a    b
0  1  NaN
1  2  NaN
2  3  5.0
2
```

Built in

Empty cells and the text NA already become NaN.

Your own marker

na_values= adds more symbols. Here the dash - is also read as missing.

~3 min

Pick Columns and Rows

Load only what you need.

```
df = pd.read_csv("people.csv",
                 usecols=["name", "city"],
                 nrows=2)
print(df)
```

Output

```
   name   city
0  Asha   Pune
1  Ravi  Delhi
```

usecols

Reads only the listed columns. This saves memory.

nrows

Reads only the first rows. Good for a quick look at a big file.

~4 min

Reading in Chunks

Handle a file too big for memory.

```
total = 0
for part in pd.read_csv("nums.csv", chunksize=4):
    print(len(part))
    total += part["x"].sum()
print(total)
```

Output

```
4
4
2
55
```

chunksize

Gives you small pieces of 4 rows each. The last piece may be smaller.

Loop and combine

Process each piece, then add up the results. Here 1 to 10 adds to 55.

~3 min

Writing a CSV File

Save a DataFrame as text.

```
df = pd.read_csv("people.csv")
text = df.to_csv(index=False, sep="|")
print(text)
df.to_csv("out.csv", index=False)
```

Output

```
name|age|city
Asha|21|Pune
Ravi|25|Delhi
Meena|23|Goa
```

index=False

Stops the row numbers from being written as an extra column.

sep and path

sep= changes the divider. A file name saves it. No name returns text.

~4 min

Working With JSON

A common format for web data.

```
one = df.head(1)
print(one.to_json(orient="records"))
df.to_json("p.json", orient="records")
print(pd.read_json("p.json").shape)
```

Output

```
[{"name":"Asha","age":21,"city":"Pune"}]
(3, 3)
```

orient

orient="records" writes one object for each row.

Round trip

Write with to_json, read back with read_json. The shape stays the same.

~3 min

Working With Excel

Read and write .xlsx sheets.

```
df.to_excel("p.xlsx", index=False)
back = pd.read_excel("p.xlsx", sheet_name=0)
print(back.shape)
print(back.columns.tolist())
```

Output

```
(3, 3)
['name', 'age', 'city']
```

sheet_name

Choose a sheet by its name or by its number. 0 is the first sheet.

Needs openpyxl

Install it once with pip install openpyxl. pandas uses it behind the scenes.

~3 min

Saving With Pickle

Keep a DataFrame exactly as it is.

```
df.to_pickle("p.pkl")
back = pd.read_pickle("p.pkl")
print(back.equals(df))
```

Output

```
True
```

Exact copy

Data types and index come back unchanged. Good for saving work in progress.

Trust the file

Loading a pickle can run code. Only open pickle files you made yourself.

~5 min

Working With a Database

Store tables and query them with SQL.

```
import sqlite3
con = sqlite3.connect(":memory:")
df.to_sql("people", con, index=False)
q = "SELECT name, age FROM people WHERE age > 22"
print(pd.read_sql(q, con))
```

Output

```
   name  age
0  Ravi   25
1  Meena   23
```

to_sql

Writes a DataFrame as a table. sqlite3 comes with Python.

read_sql

Runs a SQL query and returns the answer as a DataFrame.

~3 min

Data Loading Cheat Sheet

Match the task to the tool.

Task	Tool
Read a CSV file	`read_csv`
File has no header	`header=None, names=`
Treat symbols as missing	`na_values=`
Read a big file	`chunksize=`
Write a CSV file	`to_csv(index=False)`
Excel, JSON, database	`read_excel, read_json, read_sql`

Check shape and head() right after loading, before you do anything else.

PYTHON FOR DATA · 40 MIN

File Formats

Reading and writing data in text and binary formats · Unit 4

Text formats
Readable by people
`read_table`

Skipping lines
Clean messy files
`skiprows`

JSON and HTML
Text from the web
`read_html`

Binary formats
Compact and fast
`to_pickle`

~3 min

Reading and Writing Data Files

Four steps from a file to a DataFrame and back.

1

Text

Plain files you can open in Notepad.

`read_csv`

2

Clean

Skip lines and split columns correctly.

`sep skiprows`

3

Write

Save a DataFrame as text.

`to_csv`

4

Binary

Compact files made for programs.

`to_pickle`

Text files are easy to read and share. Binary files are smaller and faster.

~3 min

Text and Binary Formats

Where each format belongs.

Format	Kind	Read	Write
CSV or delimited	Text	`read_csv`	`to_csv`
JSON	Text	`read_json`	`to_json`
HTML table	Text	`read_html`	`to_html`
Pickle	Binary	`read_pickle`	`to_pickle`
Excel	Binary	`read_excel`	`to_excel`

Text formats can be opened in any editor. Binary formats need a program to read them.

~4 min

Delimited Text

Split columns at any separator.

```
df = pd.read_table("pipe.txt", sep="|")
print(df)
```

Output

```
   name  age   city
0  Asha   21   Pune
1  Ravi   25  Delhi
```

read_table

Like read_csv, but the default separator is a tab.

sep

sep= names the divider. Here the columns are split at the | symbol.

~4 min

Spaces Between Columns

Use a pattern when the gaps are uneven.

```
df = pd.read_table("space.txt", sep=r"\s+")
print(df)
print(df.shape)
```

Output

```
   a  b  c
0  1  2  3
1  4  5  6
(2, 3)
```

Uneven gaps

The file has one, two or three spaces between values.

The pattern \s+

Means one or more spaces. Every gap becomes a single split.

~4 min

Skipping Lines

Ignore notes and extra header lines.

```
a = pd.read_csv("notes.csv", comment="#")
b = pd.read_csv("top.csv", skiprows=1)
print(a.shape, b.shape)
print(b.columns.tolist())
```

Output

```
(2, 2) (2, 2)
['name', 'score']
```

comment

Lines that start with # are dropped. Useful for notes in a file.

skiprows

skiprows=1 skips the first line. The next line becomes the header.

~4 min

Writing Text Files

Control what to_csv saves.

```
df = pd.read_csv("people.csv")
df.loc[1, "city"] = None
text = df.to_csv(index=False, na_rep="NULL",
                 columns=["name", "city"],
                 header=False)
print(text)
```

Output

```
Asha,Pune
Ravi,NULL
Meena,Goa
```

na_rep

Chooses the text written for a missing value.

columns and header

columns= picks what to save. header=False leaves out the title row.

~4 min

JSON Text

Use the json module for nested data.

```
import json
txt = '{"name":"Asha","scores":[80,90]}'
obj = json.loads(txt)
print(obj["scores"])
print(pd.DataFrame(obj))
```

Output

```
[80, 90]
   name  scores
0  Asha      80
1  Asha      90
```

json.loads

Turns JSON text into a Python dictionary. json.dumps goes the other way.

To a DataFrame

The list of scores becomes two rows, and the name repeats.

~4 min

HTML Tables

Read tables from a web page.

```
tables = pd.read_html("page.html")
print(len(tables))
print(tables[0])
```

Output

```
1
   name  age
0  Asha   21
1  Ravi   25
```

A list of tables

read_html returns one DataFrame for every table on the page.

Needs lxml

Install it with pip install lxml. The page can be a file or a web address.

~4 min

Text Is Stored as Bytes

Every file is bytes underneath.

```
raw = open("people.csv", "rb").read()
print(raw[:4])
print(len(raw))
print(raw.decode("utf-8").splitlines()[0])
```

Output

```
b'name'
54
name,age,city
```

Binary mode

"rb" reads raw bytes. The b before the quote marks a bytes value.

Decode

decode turns bytes into text. Text files use an encoding such as utf-8.

~3 min

Binary: Pickle

Keep a DataFrame exactly as it is.

```
df.to_pickle("p.pkl")
back = pd.read_pickle("p.pkl")
print(back.equals(df))
```

Output

```
True
```

Exact copy

Data types and index come back unchanged. Good for work in progress.

Trust the file

Loading a pickle can run code. Only open pickle files you made yourself.

~3 min

Binary: Excel

Read and write .xlsx sheets.

```
df.to_excel("p.xlsx", index=False)
back = pd.read_excel("p.xlsx", sheet_name=0)
print(back.shape)
print(back.columns.tolist())
```

Output

```
(3, 3)
['name', 'age', 'city']
```

sheet_name

Choose a sheet by its name or by its number. 0 is the first sheet.

Needs openpyxl

Install it once with pip install openpyxl. pandas uses it behind the scenes.

~3 min

More Binary Formats

Fast and small for large tables.

Format	Needs	Read	Write
Parquet	`pyarrow`	`read_parquet`	`to_parquet`
Feather	`pyarrow`	`read_feather`	`to_feather`
HDF5	`tables`	`read_hdf`	`to_hdf`

Install the library with pip first. These formats are usually smaller and faster than CSV.

~3 min

File Formats Cheat Sheet

Match the task to the tool.

Task	Tool
Split at a symbol	`read_table(sep=)`
Uneven spaces	`sep=r"\s+"`
Skip notes and top lines	`comment=, skiprows=`
Save text output	`to_csv(na_rep=, header=)`
Save an exact copy	`to_pickle`
Fast big tables	`to_parquet, to_hdf`

Open a text file in an editor first. Then pick the read function that matches it.

PYTHON FOR DATA · 40 MIN

HTML, Web APIs and Databases

Interacting with HTML, web APIs and databases · Unit 4

HTML pages
Tables and tags
`read_html`

Web APIs
JSON from a server
`requests.get`

SQL queries
Ask a database
`cur.execute`

pandas and SQL
Tables in and out
`read_sql`

~3 min

Three Sources of Data

A web page, an API and a database.

1

Fetch

Get a page or API reply from a server.

`requests.get`

2

Parse

Pull out tables, tags and JSON.

`read_html .json()`

3

Store

Keep rows safely in a database.

`INSERT`

4

Query

Ask for exactly what you need.

`read_sql`

All three end the same way: the data becomes a pandas DataFrame.

~4 min

Tables From HTML

One call reads every table on a page.

```
tables = pd.read_html("page.html")
print(len(tables))
print(tables[0])
print(tables[0]["price"].sum())
```

Output

```
1
   item  price
0   Pen     10
1  Book     50
60
```

A list of tables

read_html returns one DataFrame for each table. Pick one by its position.

Needs lxml

Install it with pip install lxml. The source can be a file or a web address.

~4 min

Tags and Links

Use BeautifulSoup for other parts of a page.

```
from bs4 import BeautifulSoup
html = open("page.html").read()
soup = BeautifulSoup(html, "html.parser")
print(soup.h1.text)
links = soup.find_all("a")
print([a["href"] for a in links])
print([a.text for a in links])
```

Output

```
Prices
['/a.html', '/b.html']
['First', 'Second']
```

find_all

Returns every tag of one kind, such as all a tags for links.

Read parts

a["href"] gives the address. .text gives the words shown on the page.

~4 min

Calling a Web API

Ask a server for data.

```
import requests
url = "https://example.com/api/items"
r = requests.get(url)
print(r.status_code)
data = r.json()
print(list(data.keys()))
```

Output

```
200
['count', 'items']
```

Status code

200 means the request worked. 404 means the page was not found.

json()

Turns the JSON reply into a Python dictionary you can use right away.

~4 min

API Data in a DataFrame

Turn the reply into a table.

```
df = pd.DataFrame(data["items"])
print(df)
print(df["price"].mean())
r = requests.get(url, params={"limit": 2})
print(r.url)
```

Output

```
   id  name  price
0   1   Pen     10
1   2  Book     50
2   3   Bag    300
120.0
https://example.com/api/items?limit=2
```

List to rows

Each dictionary in the list becomes one row of the table.

params

params= adds options to the address, such as ?limit=2.

~4 min

Using a Database

Connect, create, insert and read.

```
import sqlite3
con = sqlite3.connect(":memory:")
cur = con.cursor()
cur.execute("CREATE TABLE items (id, name, price)")
cur.execute("INSERT INTO items VALUES (1,'Pen',10)")
con.commit()
cur.execute("SELECT * FROM items")
print(cur.fetchall())
```

Output

```
[(1, 'Pen', 10)]
```

cursor

The cursor sends SQL commands and holds the answer.

commit

commit saves your changes. fetchall returns the rows as tuples.

~4 min

Safe Queries

Use ? instead of pasting values into SQL.

```
sql = "INSERT INTO items VALUES (?, ?, ?)"
rows = [(2, "Book", 50), (3, "Bag", 300)]
cur.executemany(sql, rows)
q = "SELECT name FROM items WHERE price > ?"
cur.execute(q, (20,))
print(cur.fetchall())
```

Output

```
[('Book',), ('Bag',)]
```

Placeholders

Each ? is filled from the tuple. This blocks SQL injection.

executemany

Runs the same command for many rows in one call.

~4 min

pandas and SQL Together

Move whole tables in and out.

```
new = pd.DataFrame({"id": [4], "name": ["Cap"],
                    "price": [80]})
new.to_sql("items", con, if_exists="append",
           index=False)
q = "SELECT name, price FROM items"
print(pd.read_sql(q, con))
```

Output

```
   name  price
0   Pen     10
1  Book     50
2   Bag    300
3   Cap     80
```

to_sql

if_exists="append" adds the rows to the table that already exists.

read_sql

Runs a query and returns a DataFrame, ready for analysis.

~3 min

Web and Database Cheat Sheet

Match the task to the tool.

Task	Tool
Read tables from a page	`read_html`
Pick tags and links	`soup.find_all`
Call an API	`requests.get(url, params=)`
Read a JSON reply	`r.json()`
Run SQL safely	`cur.execute(sql, values)`
DataFrame and SQL	`read_sql, to_sql`

Call commit after INSERT or UPDATE, or the change is not saved.

PYTHON FOR DATA · 40 MIN

Data Wrangling

Clean, transform, merge and reshape · Unit 4

Clean
Fix messy values
`dropna`

Transform
Make new columns
`map` `cut`

Merge
Join tables
`merge` `concat`

Reshape
Wide and long
`melt` `pivot`

~3 min

Four Steps of Data Wrangling

Turn raw data into a table ready for analysis.

1

Clean

Fix missing values, repeats and wrong types.

`dropna fillna`

2

Transform

Build new columns and groups.

`map cut`

3

Merge

Join tables on a key or stack them.

`merge concat`

4

Reshape

Switch between wide and long tables.

`melt pivot`

Most analysis time goes into wrangling, so these four steps matter.

~4 min

Clean: Missing Values

Find them, drop them or fill them.

```
df = pd.DataFrame({
    "name": ["Asha", "Ravi", "Meena"],
    "age": [21, None, 23]})
print(df.isna().sum()["age"])
print(df.dropna().shape)
filled = df.fillna({"age": df["age"].mean()})
print(filled["age"].tolist())
```

Output

```
1
(2, 2)
[21.0, 22.0, 23.0]
```

Find and drop

isna marks missing cells. dropna removes every row that has one.

Fill

fillna puts a value in. Here the mean of the column, 22, is used.

~4 min

Clean: Repeats and Types

Tidy text, change types, remove copies.

```
df = pd.DataFrame({
    "name": [" Asha", "Ravi", "Ravi"],
    "age": ["21", "25", "25"]})
df["name"] = df["name"].str.strip()
df["age"] = df["age"].astype(int)
df = df.drop_duplicates()
print(df)
```

Output

```
   name  age
0  Asha   21
1  Ravi   25
```

str.strip

Removes extra spaces. Without it, " Asha" and "Asha" look different.

astype and drop_duplicates

astype changes the type. drop_duplicates keeps one copy of each row.

~4 min

Transform: New Columns

Create columns from old ones.

```
df = pd.DataFrame({"name": ["Asha", "Ravi"],
                   "marks": [80, 45]})
df["pass"] = df["marks"] >= 50
df["pct"] = df["marks"] / 100
df["dept"] = df["name"].map({"Asha": "IT",
                             "Ravi": "CS"})
print(df)
```

Output

```
   name  marks   pass   pct dept
0  Asha     80   True  0.80   IT
1  Ravi     45  False  0.45   CS
```

Maths on columns

A comparison or a division works on the whole column at once.

map

map swaps each value using a dictionary. Values not in it become NaN.

~4 min

Transform: Groups and Dummies

Bins and yes or no columns.

```
ages = pd.Series([5, 25, 70])
groups = pd.cut(ages, [0, 18, 60, 100],
                labels=["child", "adult", "senior"])
print(groups.tolist())
d = pd.get_dummies(pd.Series(["a", "b", "a"]))
print(d.astype(int).values.tolist())
```

Output

```
['child', 'adult', 'senior']
[[1, 0], [0, 1], [1, 0]]
```

cut

Sorts numbers into bins. Here 0 to 18 is child, 18 to 60 is adult.

get_dummies

Makes one 0 or 1 column for each category. Models need numbers.

~4 min

Merge: Join on a Key

Combine two tables with a shared column.

```
people = pd.DataFrame({
    "id": [1, 2, 3],
    "name": ["Asha", "Ravi", "Meena"]})
scores = pd.DataFrame({
    "id": [1, 2, 4], "score": [80, 70, 60]})
print(pd.merge(people, scores, on="id"))
```

Output

```
   id  name  score
0   1  Asha     80
1   2  Ravi     70
```

on

on= names the key column that both tables share.

Inner by default

Only ids found in both tables are kept. Ids 3 and 4 are dropped.

~4 min

Merge: Keep More Rows

Choose with how=.

```
print(pd.merge(people, scores, on="id",
               how="left"))
print(pd.merge(people, scores, on="id",
               how="outer").shape)
```

Output

```
   id   name  score
0   1   Asha   80.0
1   2   Ravi   70.0
2   3  Meena    NaN
(4, 3)
```

left and right

left keeps every row of the first table. Missing matches become NaN.

outer

outer keeps every row from both. Four ids give four rows.

~3 min

Merge: Stack With concat

Add rows or add columns.

```
a = pd.DataFrame({"x": [1, 2]})
b = pd.DataFrame({"x": [3]})
rows = pd.concat([a, b], ignore_index=True)
print(rows["x"].tolist())
cols = pd.concat([a, a], axis=1)
print(cols.shape)
```

Output

```
[1, 2, 3]
(2, 2)
```

Add rows

concat stacks tables downward. ignore_index=True renumbers the rows.

Add columns

axis=1 places the tables side by side.

~4 min

Reshape: Wide to Long

melt turns columns into rows.

```
wide = pd.DataFrame({
    "name": ["Asha", "Ravi"],
    "math": [80, 70], "art": [60, 90]})
long = wide.melt(id_vars="name",
                 var_name="subject",
                 value_name="mark")
print(long)
```

Output

```
   name subject  mark
0  Asha    math    80
1  Ravi    math    70
2  Asha     art    60
3  Ravi     art    90
```

id_vars

Columns that stay as they are. Here the name stays.

var_name and value_name

Name the new columns that hold the old headers and the values.

~4 min

Reshape: Long to Wide

pivot turns rows back into columns.

```
back = long.pivot(index="name", columns="subject",
                  values="mark")
print(back)
```

Output

```
subject  art  math
name
Asha      60    80
Ravi      90    70
```

pivot

index= gives the rows, columns= gives the headers, values= fills the cells.

pivot_table

Use pivot_table when the same pair repeats. It adds them up or averages them.

~3 min

Data Wrangling Cheat Sheet

Match the task to the tool.

Task	Tool
Missing values	`isna, dropna, fillna`
Repeats and types	`drop_duplicates, astype`
Bins and dummies	`cut, get_dummies`
Join by a key	`merge(on=, how=)`
Stack tables	`concat`
Wide and long	`melt, pivot`

Check shape and head() after every step, so mistakes show up early.

PYTHON FOR DATA · 40 MIN

Combining and Reshaping

Merging data sets, reshaping and pivoting · Unit 4

Merge
Join on keys
`merge` `join`

Concatenate
Stack or patch
`concat`

Stack
Rows and columns
`stack` `unstack`

Pivot
Long and wide
`pivot` `melt`

~3 min

Combine, Then Reshape

Two groups of tools for working with several tables.

1

Merge

Match rows of two tables by a key.

`merge join`

2

Concatenate

Stack tables or fill gaps from another.

`concat`

3

Stack

Move labels between rows and columns.

`stack unstack`

4

Pivot

Switch between long and wide tables.

`pivot melt`

Combining makes one big table. Reshaping changes its layout.

~4 min

Merge: Different Key Names

Match columns that are named differently.

```
emp = pd.DataFrame({
    "emp_id": [1, 2, 3],
    "name": ["Asha", "Ravi", "Meena"]})
pay = pd.DataFrame({
    "id": [1, 2, 4], "salary": [50, 60, 70]})
print(pd.merge(emp, pay, left_on="emp_id",
               right_on="id"))
```

Output

```
   emp_id  name  id  salary
0       1  Asha   1      50
1       2  Ravi   2      60
```

left_on and right_on

Name the key in each table when the two names differ.

Both keys stay

The result keeps emp_id and id. Drop one with drop(columns=...).

~4 min

Merge: Clashing Columns

Add suffixes when names repeat.

```
a = pd.DataFrame({"k": [1, 2], "v": [10, 20]})
b = pd.DataFrame({"k": [1, 2], "v": [30, 40]})
m = pd.merge(a, b, on="k",
             suffixes=("_a", "_b"))
print(m)
```

Output

```
   k  v_a  v_b
0  1   10   30
1  2   20   40
```

suffixes

Both tables have a column v. The suffixes tell the two apart.

Several keys

on=["k", "g"] keeps a row only when every key matches.

~4 min

Merge on the Index

merge with index flags, or join.

```
left = pd.DataFrame({"a": [1, 2]}, index=["x", "y"])
right = pd.DataFrame({"b": [3, 4]}, index=["y",
"z"])
print(pd.merge(left, right, left_index=True,
               right_index=True))
print(left.join(right, how="outer"))
```

Output

```
   a  b
y  2  3
     a    b
x  1.0  NaN
y  2.0  3.0
z  NaN  4.0
```

left_index, right_index

Use the row labels as the key. Only label y is in both tables.

join

A shortcut for index merges. Its default is how="left".

~4 min

Concatenate Tables

Stack tables along an axis.

```
a = pd.DataFrame({"x": [1, 2]})
b = pd.DataFrame({"x": [3]})
print(pd.concat([a, b], keys=["a", "b"]))
print(pd.concat([a, b], ignore_index=True).shape)
```

Output

```
     x
a 0  1
  1  2
b 0  3
(3, 1)
```

keys

Adds an outer label so you know which table each row came from.

axis=1

pd.concat([a, b], axis=1) places tables side by side, matched by index.

~4 min

Fill Gaps With combine_first

Patch missing values from another table.

```
df1 = pd.DataFrame({"x": [1, None],
                    "y": [None, 4]})
df2 = pd.DataFrame({"x": [9, 9], "y": [8, 8]})
print(df1.combine_first(df2))
```

Output

```
     x    y
0  1.0  8.0
1  9.0  4.0
```

Keep first, fill second

Values in df1 stay. Only the missing cells take df2 values.

Aligned by label

Rows and columns are matched by their labels, not by position.

~4 min

Reshape: stack

Columns become an inner row level.

```
data = pd.DataFrame({
    "math": [80, 70], "art": [60, 90]},
    index=["Asha", "Ravi"])
s = data.stack()
print(s)
```

Output

```
Asha  math    80
      art     60
Ravi  math    70
      art     90
dtype: int64
```

stack

Moves the column labels into the row index. The result is a Series.

Two levels

Each value now has two labels: the name and the subject.

~4 min

Reshape: unstack

Move a row level back to columns.

```
idx = pd.MultiIndex.from_tuples(
    [("a", "x"), ("a", "y"), ("b", "x")])
s = pd.Series([1, 2, 3], index=idx)
print(s.unstack())
print(s.unstack(fill_value=0))
```

Output

```
     x    y
a  1.0  2.0
b  3.0  NaN
   x  y
a  1  2
b  3  0
```

Missing pairs

Pair (b, y) does not exist, so unstack leaves NaN.

fill_value and level

fill_value=0 fills the gap. level=0 unstacks the outer level instead.

~4 min

Pivot: Long to Wide

Rows become columns.

```
long = pd.DataFrame({
    "name": ["Asha", "Asha", "Ravi", "Ravi"],
    "subject": ["math", "art", "math", "art"],
    "mark": [80, 60, 70, 90]})
wide = long.pivot(index="name", columns="subject",
                  values="mark")
print(wide)
```

Output

```
subject  art  math
name
Asha      60    80
Ravi      90    70
```

Three arguments

index gives the rows, columns gives the headers, values fills cells.

Unique pairs

Each name and subject pair must appear once, or pivot raises an error.

~4 min

Pivot With Repeats

pivot_table adds up repeated pairs.

```
sales = pd.DataFrame({"city": ["A", "A", "B"],
    "qty": [2, 3, 4]})
pt = sales.pivot_table(index="city", values="qty",
                       aggfunc="sum")
print(pt)
```

Output

```
      qty
city
A       5
B       4
```

aggfunc

Chooses how repeats combine, such as sum, mean or count.

Why not pivot

pivot stops with a ValueError when the same pair appears twice.

~4 min

Reshape: melt

Wide to long, choosing the columns.

```
w = pd.DataFrame({"id": [1, 2], "math": [80, 70],
                  "art": [60, 90], "pe": [1, 2]})
print(w.melt(id_vars="id",
             value_vars=["math", "art"]))
```

Output

```
   id variable  value
0   1     math     80
1   2     math     70
2   1      art     60
3   2      art     90
```

value_vars

Only the listed columns are melted. The pe column is left out.

Default names

The new columns are called variable and value unless you rename them.

~3 min

Combine and Reshape Cheat Sheet

Match the task to the tool.

Task	Tool
Join on columns	`merge(on=, left_on=, right_on=)`
Join on the index	`join, merge(left_index=True)`
Stack tables	`concat`
Fill gaps from another table	`combine_first`
Rows to columns	`unstack, pivot, pivot_table`
Columns to rows	`stack, melt`

Check shape and head() after every merge to catch lost or repeated rows.

PYTHON FOR DATA · 40 MIN

Data Transformation and String Manipulation

Cleaning values, binning and working with text · Unit 4

Clean

Drop repeats

`drop_duplicates`

Map

Swap and rename

`map` `replace`

Bin

Numbers to groups

`cut` `qcut`

Strings

Text, per value

`.str methods`

Transform, Then Handle Text

~3 min

Two groups of tools for cleaning a column.

1

Clean

Find and drop repeated rows.

`duplicated` `drop_duplicates`

2

Map

Swap values and rename labels.

`map` `replace` `rename`

3

Bin

Turn numbers into groups.

`cut` `qcut`

4

Strings

Run text methods on a whole column.

`.str` `split` `extract`

Transforming changes values. String methods change text.

Remove Duplicates

~4 min

Find and drop repeated rows.

```
df = pd.DataFrame({
    "name": ["Asha", "Ravi", "Asha", "Asha"],
    "city": ["Pune", "Goa", "Pune", "Delhi"]})
print(df.duplicated().tolist())
print(df.drop_duplicates())
```

Output

```
[False, False, True, False]
   name   city
0  Asha   Pune
1  Ravi    Goa
3  Asha  Delhi
```

duplicated

Marks a row True when an earlier row matches it in every column.

subset and keep

subset="name" compares one column. keep="last" keeps the final repeat.

Map and Replace

~4 min

Swap values with a dictionary.

```
s = pd.Series(["a", "b", "c", "a"])
print(s.map({"a": "x", "b": "y"}).tolist())
print(s.replace({"a": "x", "b": "y"}).tolist())
```

Output

```
['x', 'y', nan, 'x']
['x', 'y', 'c', 'x']
```

map

Looks up every value. Anything missing from the dictionary becomes NaN.

replace

Changes only the listed values. replace(-99, np.nan) marks a sentinel as missing.

Rename Labels

~4 min

Change index and column names.

```
d = pd.DataFrame({"a": [1, 2], "b": [3, 4]},
                 index=["x", "y"])
print(d.rename(columns={"a": "A"},
               index=str.upper))
```

Output

```
   A  b
X  1  3
Y  2  4
```

Dict or function

A dict renames chosen labels. A function such as str.upper changes all of them.

Not in place

rename returns a new table. Assign it back to keep the change.

Bin With cut

~4 min

Turn numbers into labelled groups.

```
ages = pd.Series([5, 17, 30, 70])
bins = [0, 18, 60, 100]
tags = ["kid", "adult", "senior"]
print(pd.cut(ages, bins, labels=tags).tolist())
print(pd.qcut([1, 2, 3, 4], 2,
              labels=["low", "high"]).tolist())
```

Output

```
['kid', 'kid', 'adult', 'senior']
['low', 'low', 'high', 'high']
```

cut

Splits at edges you choose. Bins close on the right, so (0, 18] includes 18.

qcut

Splits by quantiles, so each bin gets about the same number of values.

String Basics: .str

~4 min

One method call for the whole column.

```
t = pd.Series([" Asha Rao ", "ravi KUMAR", None])
print(t.str.strip().str.title().tolist())
print(t.str.len().tolist())
```

Output

```
['Asha Rao', 'Ravi Kumar', nan]
[10.0, 10.0, nan]
```

.str accessor

Applies a text method to every value. A missing value stays NaN.

Chain methods

strip, lower, upper and title can be chained in a single line.

Split and Search

~4 min

Break text apart and test it.

```
n = pd.Series(["Asha Rao", "Ravi Kumar"])
print(n.str.split(" ", expand=True))
print(n.str.contains("ao").tolist())
print(n.str.startswith("A").tolist())
```

Output

```
      0      1
0  Asha    Rao
1  Ravi  Kumar
[True, False]
[True, False]
```

expand=True

Returns a table with one column per piece, not a list in each cell.

contains, startswith

Give True or False per value. Use them as a filter on rows.

Patterns With Regex

~4 min

Replace, capture and find.

```
p = pd.Series(["a1-22", "b3-4"])
print(p.str.replace(r"\d+", "#", regex=True).tolist())
print(p.str.extract(r"([a-z])(\d)"))
print(p.str.findall(r"\d+").tolist())
```

Output

```
['a#-#', 'b#-#']
   0  1
0  a  1
1  b  3
[['1', '22'], ['3', '4']]
```

Patterns

contains, replace, extract and findall accept patterns. Pass regex=True to replace.

extract

Each group in parentheses becomes a column. No match gives NaN.

Join and Dummies

~4 min

Build text and indicator columns.

```
a = pd.Series(["a", "b"])
print(a.str.cat(["x", "y"], sep="-").tolist())
print(a.str.cat(sep="+"))
print(pd.Series(["a|b", "c"]).str.get_dummies("|"))
```

Output

```
['a-x', 'b-y']
a+b
   a  b  c
0  1  1  0
1  0  0  1
```

cat

Joins strings. With only sep, it joins the whole Series into one string.

get_dummies

Splits on a separator and makes a 0 and 1 column for each label.

Transform and Strings Cheat Sheet

~3 min

Match the task to the tool.

Task	Tool
Drop repeated rows	`duplicated, drop_duplicates`
Swap values	`map, replace`
Rename labels	`rename`
Numbers to groups	`cut, qcut`
Clean text	`str.strip, lower, upper, title`
Split and search	`str.split, contains, startswith`
Patterns	`str.replace, extract, findall`
Join text, make flags	`str.cat, str.get_dummies`

Check for NaN with isna() before you use .str on a column.

PYTHON FOR DATA · 40 MIN

Data Aggregation and Group Operations

GroupBy mechanics, aggregation and transformations · Unit 4

Split

Group the rows

`groupby`

Aggregate

One row per group

`agg` `sum` `mean`

Transform

Keep every row

`transform`

Apply

Custom per group

`apply` `filter`

Split, Apply, Combine

~3 min

One idea behind every group operation.

1 Split

Divide the rows into groups by a key.

`groupby` `size`

2 Aggregate

Reduce each group to one row.

`agg` `sum` `mean`

3 Transform

Return values aligned to the rows.

`transform` `cumsum`

4 Apply

Run your own function on each group.

`apply` `filter`

Aggregation shrinks the table. Transformation keeps its shape.

GroupBy Mechanics

~4 min

Split the rows into groups by a key.

```
df = pd.DataFrame({"team": list("AABBB"),
   "role": ["dev", "ops", "dev", "dev", "ops"],
   "pay": [50, 60, 70, 80, 90]})
g = df.groupby("team")
print(g.size())
for name, part in g:
    print(name, part["pay"].tolist())
```

Output

```
team
A    2
B    3
dtype: int64
A [50, 60]
B [70, 80, 90]
```

Lazy object

groupby computes nothing yet. It waits until you call a method such as size or sum.

Iterate

Looping gives each key and its sub-table. Handy for inspecting a group.

Group by Two Keys

~4 min

Several keys make a hierarchical index.

```
df = pd.DataFrame({"team": list("AABBB"),
   "role": ["dev", "ops", "dev", "dev", "ops"],
   "pay": [50, 60, 70, 80, 90]})
tot = df.groupby(["team", "role"])["pay"].sum()
print(tot.unstack())
print(tot["B"].tolist())
```

Output

```
role  dev  ops
team
A      50   60
B     150   90
[150, 90]
```

Two index levels

tot has team as the outer level and role as the inner level.

unstack

Moves the inner level into columns. Select an outer key with tot["B"].

Aggregate With agg

~4 min

Summarize each group.

```
df = pd.DataFrame({"team": list("AABBB"),
   "role": ["dev", "ops", "dev", "dev", "ops"],
   "pay": [50, 60, 70, 80, 90]})
g = df.groupby("team")
print(g["pay"].agg(["min", "max", "mean"]))
```

Output

```
      min  max  mean
team
A      50   60  55.0
B      70   90  80.0
```

agg with a list

Each function name becomes a column. Try sum, count, first or std.

Names are fastest

A name such as "mean" uses optimized code. A lambda is slower.

Named Aggregation

~4 min

Pick the output names yourself.

```
df = pd.DataFrame({"team": list("AABBB"),
   "role": ["dev", "ops", "dev", "dev", "ops"],
   "pay": [50, 60, 70, 80, 90]})
out = df.groupby("team").agg(
    total=("pay", "sum"), people=("role", "count"))
print(out)
```

Output

```
      total  people
team
A       110       2
B       240       3
```

name=(column, func)

Each keyword sets an output column. Different columns get different functions.

count vs size

count skips NaN in a column. size counts every row in the group.

Keys and Custom Functions

~4 min

Control the index and write your own.

```
df = pd.DataFrame({"team": list("AABBB"),
   "role": ["dev", "ops", "dev", "dev", "ops"],
   "pay": [50, 60, 70, 80, 90]})
print(df.groupby("team", as_index=False)["pay"].sum())
rng = lambda s: s.max() - s.min()
print(df.groupby("team")["pay"].agg(rng))
```

Output

```
  team  pay
0    A  110
1    B  240
team
A    10
B    20
Name: pay, dtype: int64
```

as_index=False

Keeps the key as a normal column. reset_index() does the same afterwards.

Custom function

Any function that turns a group into one value works inside agg.

Transform: Keep the Shape

~4 min

Group results aligned to every row.

```
df = pd.DataFrame({"team": list("AABBB"),
   "role": ["dev", "ops", "dev", "dev", "ops"],
   "pay": [50, 60, 70, 80, 90]})
avg = df.groupby("team")["pay"].transform("mean")
df["gap"] = df["pay"] - avg
print(df[["team", "pay", "gap"]])
```

Output

```
  team  pay   gap
0    A   50  -5.0
1    A   60   5.0
2    B   70 -10.0
3    B   80   0.0
4    B   90  10.0
```

transform

Returns a result as long as the original, so it lines up row by row.

Group centering

Subtract the group mean to compare people inside their own team.

Apply: Any Function

~4 min

Run your own code on each group.

```
df = pd.DataFrame({"team": list("AABBB"),
   "role": ["dev", "ops", "dev", "dev", "ops"],
   "pay": [50, 60, 70, 80, 90]})
top2 = df.groupby("team")["pay"].apply(
    lambda s: s.nlargest(2))
print(top2)
```

Output

```
team
A     1    60
      0    50
B     4    90
      3    80
Name: pay, dtype: int64
```

apply

Runs your function on each group. It may return a value, a Series or a table.

Outer index

When a group returns several rows, the key becomes the outer index level.

Filter and Running Totals

~4 min

Drop whole groups or accumulate inside them.

```
df = pd.DataFrame({"team": list("AABBB"),
   "role": ["dev", "ops", "dev", "dev", "ops"],
   "pay": [50, 60, 70, 80, 90]})
g = df.groupby("team")
print(g.filter(lambda d: len(d) > 2))
print(g["pay"].cumsum().tolist())
```

Output

```
  team role  pay
2    B  dev   70
3    B  dev   80
4    B  ops   90
[50, 110, 70, 150, 240]
```

filter

Keeps or drops whole groups. The function must return True or False.

cumsum

Restarts for each group. rank and shift also work inside groups.

GroupBy Cheat Sheet

~3 min

Match the task to the tool.

Task	Tool
Split rows into groups	`df.groupby(key)`
Group by two keys	`groupby([k1, k2])`
Several summaries	`agg([...])`, `agg(name=(col, func))`
Keep the key as a column	`as_index=False`, `reset_index`
Group values on every row	`transform`
Any custom group step	`apply`
Drop whole groups	`filter`
Running totals, ranks	`cumsum`, `rank`

Check shape and head() after every groupby to catch lost or repeated rows.

PYTHON FOR DATA · 40 MIN

Pivot Tables and Cross Tabulation

Summary grids and frequency tables · Unit 4

Pivot

Rows by columns

`pivot_table`

Aggfunc

Pick the summary

`aggfunc` `margins`

Crosstab

Count the pairs

`crosstab`

Normalize

Counts to shares

`normalize`

Summarize in a Grid

~3 min

Two tools, one idea: rows by columns.

1

Pivot table

Summarize a column by two labels.

`pivot_table`

2

Aggregate

Choose sum, mean, count or your own.

`aggfunc` `margins`

3

Crosstab

Count how often label pairs occur.

`crosstab`

4

Normalize

Turn counts into shares.

`normalize`

pivot_table summarizes values. crosstab counts pairs by default.

Pivot Table Basics

~4 min

Summarize a column by two labels.

```
sales = pd.DataFrame({"city": list("AABBB"),
    "item": ["pen", "pen", "pen", "ink", "ink"],
    "qty": [2, 3, 4, 1, 5],
    "rev": [20, 30, 40, 10, 50]})
pt = sales.pivot_table(index="city",
    columns="item", values="qty", aggfunc="sum")
print(pt)
```

Output

```
item  ink  pen
city
A     NaN  5.0
B     6.0  4.0
```

Three roles

index gives the rows, columns gives the headers, values fills the cells.

Repeats are fine

(A, pen) appears twice, so 2 and 3 are added to 5. A missing pair gives NaN.

Choose the aggfunc

~4 min

Give each column its own summary.

```
sales = pd.DataFrame({"city": list("AABBB"),
    "item": ["pen", "pen", "pen", "ink", "ink"],
    "qty": [2, 3, 4, 1, 5],
    "rev": [20, 30, 40, 10, 50]})
g = sales.pivot_table(index="city",
    values=["qty", "rev"],
    aggfunc={"qty": "sum", "rev": "mean"})
print(g)
```

Output

```
      qty        rev
city
A       5  25.000000
B      10  33.333333
```

A dict

Gives each column its own function. A list gives every column several functions.

Names or callables

Use names such as sum, mean, count and max, or pass your own function.

Fill Gaps and Add Totals

~4 min

Tidy the grid for reading.

```
sales = pd.DataFrame({"city": list("AABBB"),
    "item": ["pen", "pen", "pen", "ink", "ink"],
    "qty": [2, 3, 4, 1, 5],
    "rev": [20, 30, 40, 10, 50]})
pt = sales.pivot_table(index="city",
    columns="item", values="qty", aggfunc="sum",
    fill_value=0, margins=True)
print(pt)
```

Output

```
item  ink  pen  All
city
A       0    5    5
B       6    4   10
All     6    9   15
```

fill_value

Replaces the NaN of a pair that never occurs, such as (A, ink).

margins

margins=True adds an All row and column. margins_name renames them.

Same as groupby

~4 min

Two routes to one grid.

```
sales = pd.DataFrame({"city": list("AABBB"),
    "item": ["pen", "pen", "pen", "ink", "ink"],
    "qty": [2, 3, 4, 1, 5],
    "rev": [20, 30, 40, 10, 50]})
a = sales.groupby(["city", "item"])["qty"].sum()
b = sales.pivot_table(index="city", columns="item",
    values="qty", aggfunc="sum")
print(a.unstack().equals(b))
```

Output

```
True
```

Two routes

groupby then unstack builds the same grid. pivot_table is the shortcut.

When to pick

Use groupby for a long result. Use pivot_table for a grid with margins or fill_value.

Cross Tabulation

~4 min

Count how often label pairs occur.

```
sales = pd.DataFrame({"city": list("AABBB"),
    "item": ["pen", "pen", "pen", "ink", "ink"],
    "qty": [2, 3, 4, 1, 5],
    "rev": [20, 30, 40, 10, 50]})
ct = pd.crosstab(sales["city"], sales["item"])
print(ct)
```

Output

```
item  ink  pen
city
A       0    2
B       2    1
```

Counts pairs

crosstab counts each pair of labels. It needs no values column.

Zero, not NaN

A pair that never occurs shows 0. Here (A, ink) is 0.

Crosstab Totals

~4 min

Add row and column totals.

```
sales = pd.DataFrame({"city": list("AABBB"),
    "item": ["pen", "pen", "pen", "ink", "ink"],
    "qty": [2, 3, 4, 1, 5],
    "rev": [20, 30, 40, 10, 50]})
ct = pd.crosstab(sales["city"], sales["item"],
    margins=True, margins_name="Total")
print(ct)
```

Output

```
item   ink  pen  Total
city
A        0    2      2
B        2    1      3
Total    2    3      5
```

margins

Adds a total for every row and column. margins_name sets the label.

Check the total

The corner cell must equal the number of rows in the source table.

Normalize to Shares

~4 min

Turn counts into proportions.

```
sales = pd.DataFrame({"city": list("AABBB"),
    "item": ["pen", "pen", "pen", "ink", "ink"],
    "qty": [2, 3, 4, 1, 5],
    "rev": [20, 30, 40, 10, 50]})
ct = pd.crosstab(sales["city"], sales["item"],
    normalize="index")
print(ct.round(2))
```

Output

```
item   ink   pen
city
A     0.00  1.00
B     0.67  0.33
```

normalize

"index" makes each row sum to 1. "columns" does the same for columns. True uses the whole table.

Percentages

Multiply by 100 for percentages. round tidies the display.

Crosstab With Values

~4 min

Summarize a column, not just count.

```
sales = pd.DataFrame({"city": list("AABBB"),
    "item": ["pen", "pen", "pen", "ink", "ink"],
    "qty": [2, 3, 4, 1, 5],
    "rev": [20, 30, 40, 10, 50]})
ct = pd.crosstab(sales["city"], sales["item"],
    values=sales["qty"], aggfunc="sum")
print(ct)
```

Output

```
item  ink  pen
city
A     NaN  5.0
B     6.0  4.0
```

values and aggfunc

Pass both together. Without them crosstab only counts.

NaN returns

With values, a missing pair is NaN again. Add .fillna(0) to fix it.

Pivot and Crosstab Cheat Sheet

~3 min

Match the task to the tool.

Task	Tool
Summary grid of a column	`pivot_table(index, columns, values)`
Choose the summary	`aggfunc: name, list or dict`
Fill gaps, add totals	`fill_value=0, margins=True`
Count label pairs	`pd.crosstab(a, b)`
Shares instead of counts	`normalize="index"`
Summarize inside crosstab	`values=, aggfunc=`
Same grid by groups	`groupby(...).unstack()`
Long to wide, no repeats	`pivot`

pivot_table sums values. crosstab counts. Check totals against the source.

PYTHON FOR DATA · 40 MIN

Data Visualization

matplotlib, seaborn and pandas plotting · Unit 4

matplotlib

Figure and Axes

`plt.subplots`

Seaborn

Statistical plots

`sns.histplot`

pandas

Plots from data

`df.plot`

Customize

Labels and saving

`ax.set` `savefig`

Three Layers of Plotting

~2 min

Each layer builds on matplotlib.

1

matplotlib

Figures and Axes with full control.

`plt.subplots` `ax.plot`

2

Seaborn

Statistical plots from a DataFrame.

`histplot` `boxplot` `heatmap`

3

pandas

Quick plots straight from your data.

`df.plot(kind=...)`

4

Customize

Titles, labels, legends and saving.

`ax.set` `legend` `savefig`

seaborn and pandas return matplotlib Axes, so you can tune any plot.

Figure and Axes

~4 min

The matplotlib object model.

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
fig, ax = plt.subplots()
ax.plot(x, [1, 4, 9, 16], marker="o")
ax.bar(x, [2, 3, 5, 4], alpha=0.4)
plt.show()
```

Plot

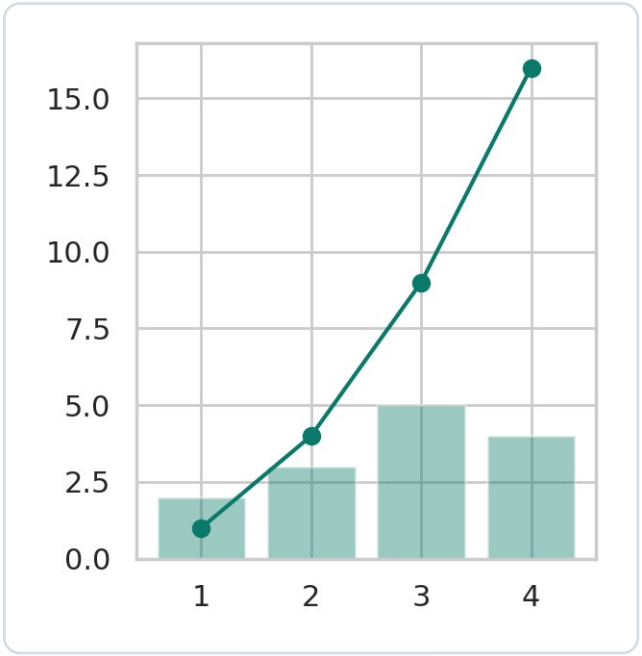


Figure and Axes

The figure is the canvas. An Axes is one plot on it.

Draw on ax

ax.plot, ax.bar, ax.scatter and ax.hist each add one layer.

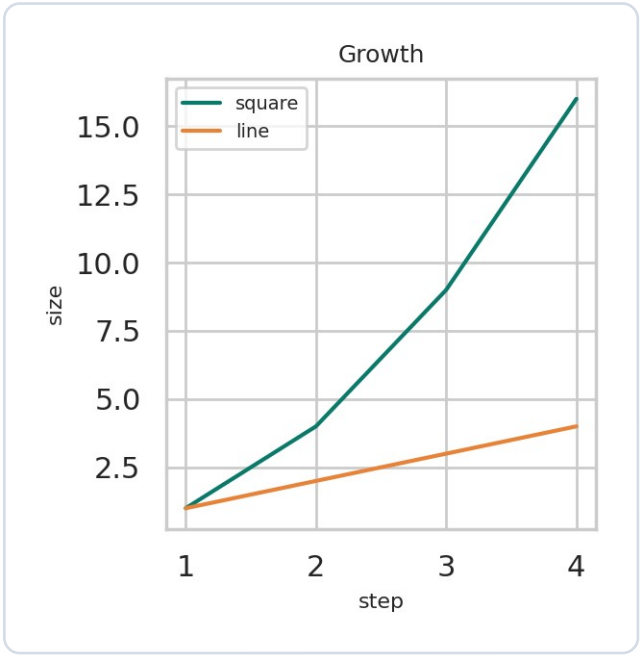
Labels and Legend

~4 min

Make the plot readable.

```
x = [1, 2, 3, 4]
fig, ax = plt.subplots()
ax.plot(x, [1, 4, 9, 16], label="square")
ax.plot(x, [1, 2, 3, 4], label="line")
ax.set(title="Growth", xlabel="step",
       ylabel="size", xticks=x)
ax.legend()
```

Plot



ax.set

Sets title, labels, limits and ticks in one call.

label, legend

Give each line a label. ax.legend() then draws the key.

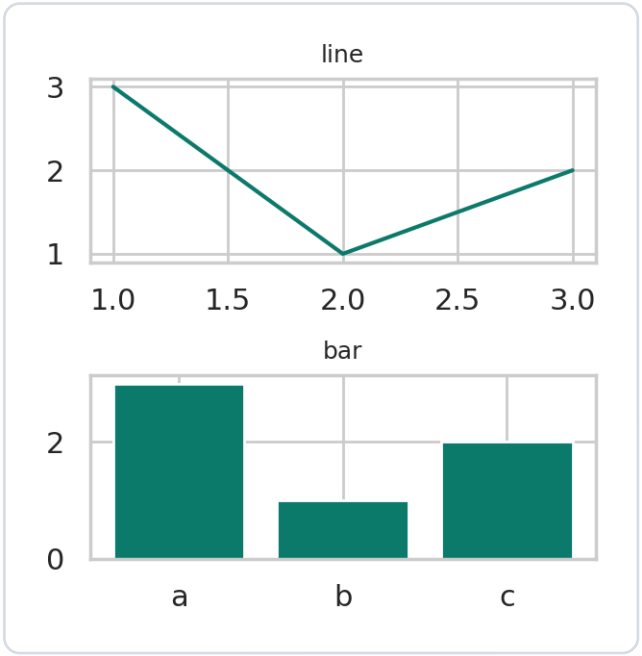
Subplots and Saving

~4 min

A grid of Axes in one figure.

```
fig, axs = plt.subplots(2, 1)
axs[0].plot([1, 2, 3], [3, 1, 2])
axs[0].set_title("line")
axs[1].bar(["a", "b", "c"], [3, 1, 2])
axs[1].set_title("bar")
fig.tight_layout()
fig.savefig("plots.png", dpi=150)
```

Plot



Array of Axes

subplots(rows, cols) returns Axes you index like a list.

tight_layout

Stops overlaps. savefig writes png, pdf or svg.

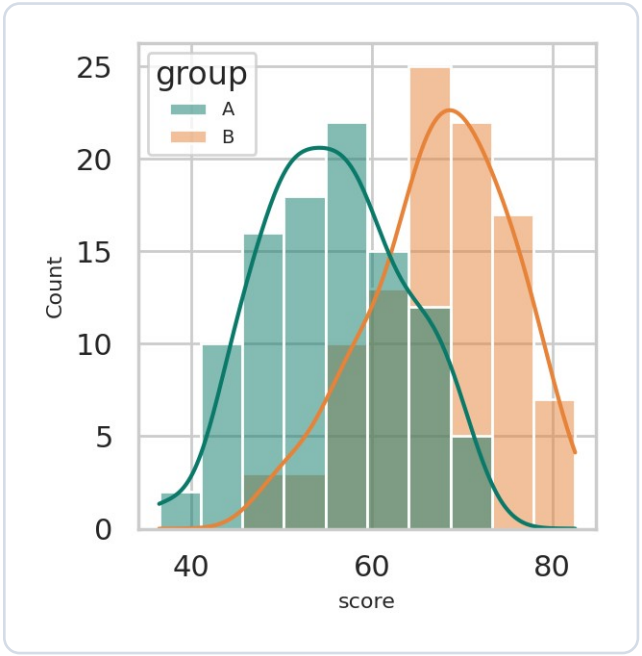
Seaborn: Distributions

~4 min

Seaborn works on a whole DataFrame.

```
import numpy as np, pandas as pd, seaborn as sns
rng = np.random.default_rng(0)
a, b = rng.normal(55, 8, 100), rng.normal(68, 8, 100)
df = pd.DataFrame({"group": ["A"] * 100 + ["B"] * 100,
    "score": np.r_[a, b]})
sns.histplot(df, x="score", hue="group",
    kde=True)
```

Plot



hue

Splits the data by a column and colours each group.

kde=True

Adds a smooth density curve over the histogram.

Seaborn: Compare Groups

~4 min

Categorical plots.

```
import numpy as np, pandas as pd, seaborn as sns
rng = np.random.default_rng(0)
a, b = rng.normal(55, 8, 100), rng.normal(68, 8, 100)
df = pd.DataFrame({"group": ["A"] * 100 + ["B"] * 100,
    "score": np.r_[a, b]})
sns.boxplot(df, x="group", y="score")
sns.stripplot(df, x="group", y="score", size=3)
```

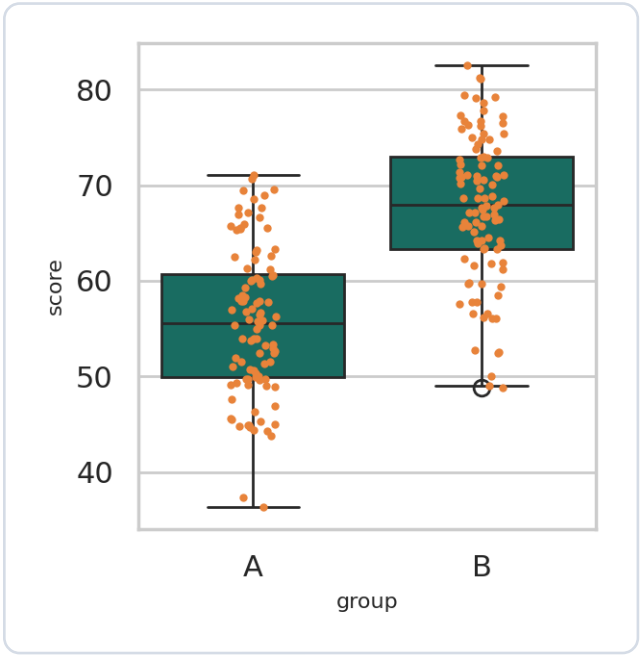
boxplot

Shows the median, quartiles and outliers of each group.

Layer plots

A stripplot on top shows every point. barplot shows means.

Plot



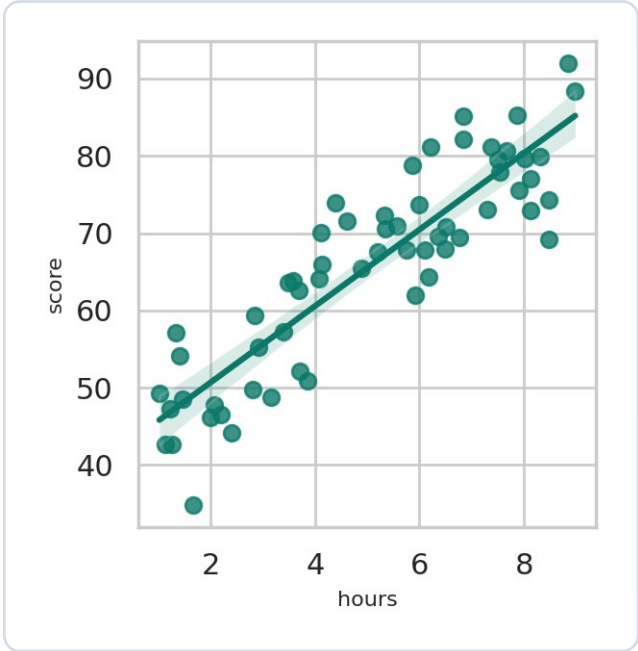
Seaborn: Relationships

~4 min

A scatter plot with a fitted line.

```
import numpy as np, pandas as pd, seaborn as sns
rng = np.random.default_rng(0)
hours = rng.uniform(1, 9, 60)
df = pd.DataFrame({"hours": hours,
    "score": 40 + 5 * hours + rng.normal(0, 6, 60)})
sns.regplot(data=df, x="hours", y="score")
```

Plot



regplot

Draws the points, a fitted line and its confidence band.

scatterplot

Use hue and size to add more columns to one plot.

Seaborn: Heatmap

~4 min

See every correlation at once.

```
import numpy as np, pandas as pd, seaborn as sns
rng = np.random.default_rng(0)
df = pd.DataFrame(rng.normal(size=(100, 3)),
    columns=["a", "b", "c"])
df["d"] = df["a"] * 2 + rng.normal(size=100)
sns.heatmap(df.corr(), annot=True, cmap="Blues")
```

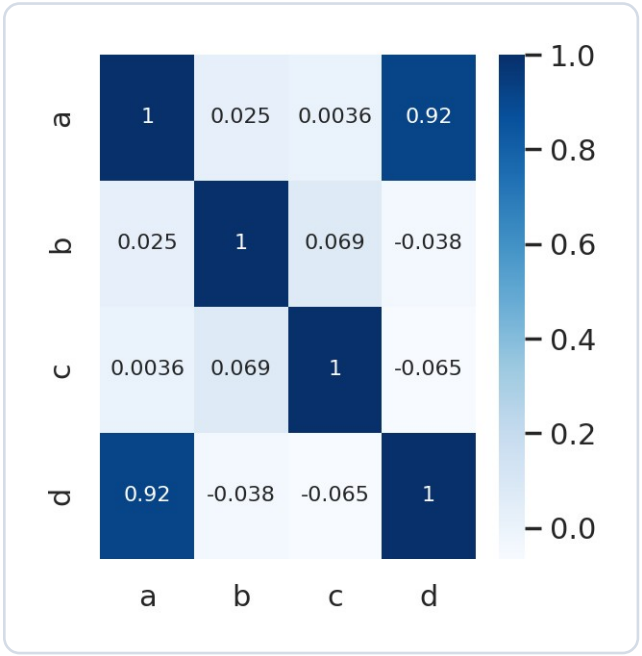
corr()

Builds the table of correlations between numeric columns.

annot, cmap

annot=True prints each value. cmap sets the colours.

Plot



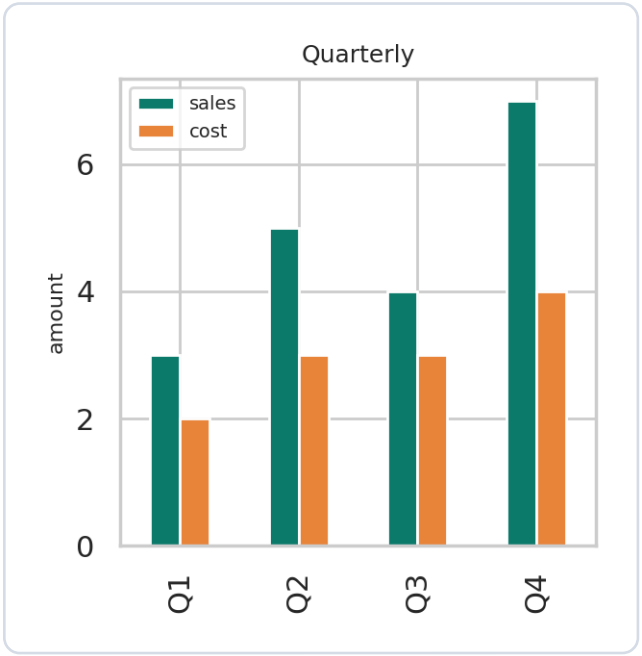
pandas: df.plot

~4 min

df.plot wraps matplotlib.

```
import pandas as pd
df = pd.DataFrame({"sales": [3, 5, 4, 7],
    "cost": [2, 3, 3, 4]},
    index=["Q1", "Q2", "Q3", "Q4"])
ax = df.plot(kind="bar", title="Quarterly")
ax.set_ylabel("amount")
```

Plot



df.plot

One series per column, with the index on the x axis.

Returns an Axes

Call set_ylabel and other matplotlib methods on ax.

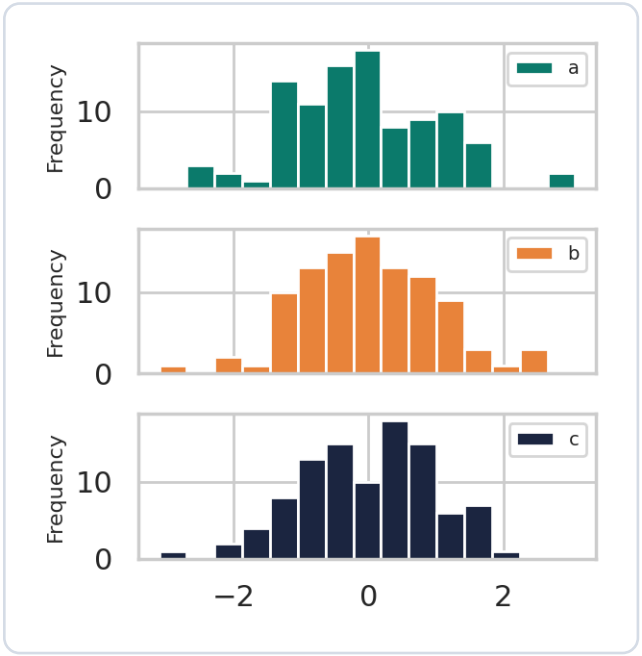
pandas: Other Kinds

~4 min

Change kind, keep the same call.

```
import numpy as np, pandas as pd
rng = np.random.default_rng(0)
df = pd.DataFrame(rng.normal(size=(100, 3)),
    columns=["a", "b", "c"])
df.plot(kind="hist", bins=15, subplots=True,
        sharex=True)
```

Plot



kind

line, bar, barh, hist, box, kde, area, scatter and pie.

subplots=True

One panel per column. scatter takes x= and y= columns.

Plotting Cheat Sheet

~2 min

Match the task to the tool.

Task	Tool
One plot on a figure	`fig, ax = plt.subplots()`
Several panels	`plt.subplots(rows, cols)`
Titles, labels, legend	`ax.set(...), ax.legend()`
Save the figure	`fig.savefig("name.png")`
Distributions	`sns.histplot, sns.kdeplot`
Compare groups	`sns.boxplot, barplot, violinplot`
Relationships	`sns.scatterplot, regplot, heatmap`
Quick plots from data	`df.plot(kind=...)`

Label every axis. Pick the plot that matches the question.

PYTHON FOR DATA · 40 MIN

Time Series and Case Studies

Dates, resampling, windows and real-world cases · Unit 4

Dates

Parse and index

`to_datetime`

Resample

Change frequency

`resample` `asfreq`

Windows

Compare with the past

`rolling` `shift`

Cases

Sales, traffic, risk

`plot` `groupby`

The Time Series Toolkit

~2 min

Dates first, then resample and roll.

1

Dates

Parse text and build date ranges.

`to_datetime` `date_range`

2

Select

Slice by date and set a frequency.

`ts["2026-02"]` `asfreq`

3

Resample

Change frequency, shift and smooth.

`resample` `shift` `rolling`

4

Case studies

Sales, web traffic and risk.

`plot` `groupby` `rolling`

Check the index is a sorted DatetimeIndex before anything else.

Dates and Timestamps

~3 min

Turn text into datetime values.

```
s = pd.Series(["2026-01-05", "2026-02-10"])
d = pd.to_datetime(s)
print(d.dt.month.tolist(), d.dt.day_name().tolist())
print(d[1] - d[0])
r = pd.date_range("2026-01-01", periods=3)
print(r.strftime("%d %b").tolist())
```

Output

```
[1, 2] ['Monday', 'Tuesday']
36 days 00:00:00
['01 Jan', '02 Jan', '03 Jan']
```

to_datetime and dt

Parses text into dates. The dt accessor reads year, month, day_name and more.

date_range

Builds a regular index from a start, periods and a freq. The default freq is daily.

Time Series Indexing

~3 min

Select by date with a datetime index.

```
idx = pd.date_range("2026-01-30", periods=5)
ts = pd.Series([1, 2, 3, 4, 5], index=idx)
print(ts["2026-02"].tolist())
print(ts["2026-01-31":"2026-02-02"].tolist())
print(ts.index.is_monotonic_increasing)
```

Output

```
[3, 4, 5]
[2, 3, 4]
True
```

Partial strings

"2026-02" selects a whole month and "2026" selects a year.

Date slices

Label slices include both ends. Keep the index sorted.

Frequencies and asfreq

~3 min

Name a step, then fill the gaps.

```
m = pd.date_range("2026-01-31", periods=3, freq="ME")
print(m.strftime("%b %d").tolist())
b = pd.date_range("2026-01-02", periods=3, freq="B")
print(b.strftime("%a").tolist())
ts = pd.Series([1, 2], index=pd.date_range(
    "2026-01-01", periods=2, freq="3D"))
print(ts.asfreq("D", method="ffill").tolist())
```

Output

```
['Jan 31', 'Feb 28', 'Mar 31']
['Fri', 'Mon', 'Tue']
[1, 1, 1, 2]
```

Frequency codes

D day, B business day, W week, ME month end and h hour.

asfreq

Re-indexes to a new frequency. method="ffill" fills the new gaps.

Resample: Downsample

~3 min

Group by time bins, then summarize.

```
idx = pd.date_range("2026-01-01", periods=6)
ts = pd.Series([1, 2, 3, 4, 5, 6], index=idx)
print(ts.resample("3D").sum().tolist())
print(ts.resample("W").mean())
```

Output

```
[6, 15]
2026-01-04    2.5
2026-01-11    5.5
Freq: W-SUN, dtype: float64
```

resample

Cuts the index into time bins. Then pick sum, mean or another aggregate.

Bin edges

Weekly bins end on Sunday by default. closed and label change this.

Resample: Upsample

~3 min

Go to a finer frequency and fill.

```
d = pd.to_datetime(["2026-01-01", "2026-01-03"])
ts = pd.Series([10, 20], index=d)
print(ts.resample("D").asfreq().tolist())
print(ts.resample("D").ffill().tolist())
print(ts.resample("D").interpolate().tolist())
```

Output

```
[10.0, nan, 20.0]
[10, 10, 20]
[10.0, 15.0, 20.0]
```

Empty slots

A finer frequency creates new dates with no value. You choose how to fill.

ffill, interpolate

ffill repeats the last value. interpolate draws a straight line.

Shift, Diff, pct_change

~3 min

Compare each row with the past.

```
idx = pd.date_range("2026-01-01", periods=3)
ts = pd.Series([100, 110, 99], index=idx)
print(ts.shift(1).tolist())
print(ts.diff().tolist())
print(ts.pct_change().round(2).tolist())
print(ts.shift(freq="D").index[0].strftime("%b %d"))
```

Output

```
[nan, 100.0, 110.0]
[nan, 10.0, -11.0]
[nan, 0.1, -0.1]
Jan 02
```

shift

Moves values down one row. The edge becomes NaN. freq= moves the dates instead.

diff, pct_change

Change and percent change from the previous row. Both give returns.

Rolling Windows

~3 min

Moving statistics over time.

```
idx = pd.date_range("2026-01-01", periods=5)
ts = pd.Series([1, 2, 3, 4, 10], index=idx)
print(ts.rolling(3).mean().round(1).tolist())
print(ts.rolling(3, min_periods=1).sum().tolist())
print(ts.rolling("2D").mean().tolist())
print(ts.expanding().max().tolist())
```

Output

```
[nan, nan, 2.0, 3.0, 5.7]
[1.0, 3.0, 6.0, 9.0, 17.0]
[1.0, 1.5, 2.5, 3.5, 7.0]
[1.0, 2.0, 3.0, 4.0, 10.0]
```

rolling(n)

A moving window of n rows. The first n-1 results are NaN.

Time windows

rolling("7D") uses calendar time. expanding() grows from the start.

Time Zones

~3 min

Attach a zone, then convert.

```
d = pd.to_datetime(["2026-03-01 09:00",
    "2026-03-01 21:00"])
ts = pd.Series([1, 2], index=d)
utc = ts.tz_localize("UTC")
ist = utc.tz_convert("Asia/Kolkata")
print(utc.index.tz, ist.index.tz)
print(ist.index[0].strftime("%H:%M %Z"))
```

Output

```
UTC Asia/Kolkata
14:30 IST
```

Naive vs aware

tz_localize attaches a zone to naive times. It does not shift them.

tz_convert

Converts an aware index to another zone. The instant stays the same.

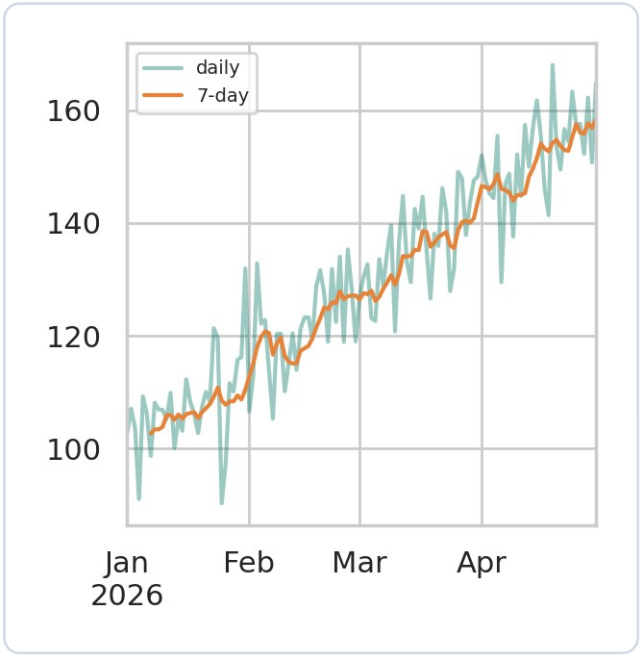
Case: Retail Sales Trend

~4 min

Smooth noisy daily sales (synthetic data).

```
import numpy as np, pandas as pd
rng = np.random.default_rng(1)
days = pd.date_range("2026-01-01", periods=120)
sales = pd.Series(100 + 0.5 * np.arange(120) +
    rng.normal(0, 8, 120), index=days)
ax = sales.plot(alpha=0.4, label="daily")
sales.rolling(7).mean().plot(ax=ax, label="7-day")
ax.legend()
```

Plot



The question

Is demand really rising, or is daily noise misleading us?

The method

A 7-day rolling mean smooths the noise and shows the trend.

Case: Web Traffic by Hour

~4 min

Find the busiest hour (synthetic data).

```
import numpy as np, pandas as pd
rng = np.random.default_rng(2)
t = pd.date_range("2026-03-01", periods=72, freq="h")
lam = 20 + 30 * (t.hour == 20)
hits = pd.Series(rng.poisson(lam), index=t)
by_hour = hits.groupby(hits.index.hour).mean()
print(by_hour.idxmax(), round(by_hour.max(), 1))
print(hits.resample("D").sum().tolist())
```

Output

```
20 44.0
[517, 475, 563]
```

The question

Which hour is busiest, so servers can be scaled up in time?

The method

Group by index.hour to average each hour. resample("D") totals each day.

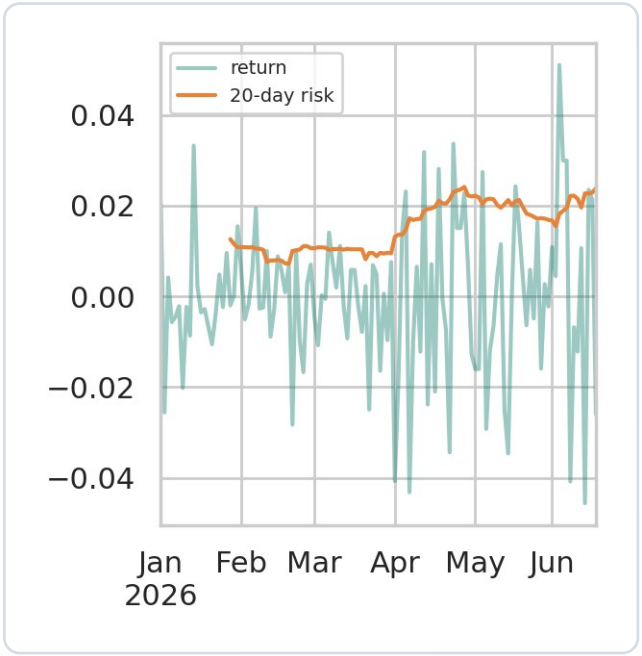
Case: Returns and Risk

~4 min

Watch risk move over time (synthetic data).

```
import numpy as np, pandas as pd
rng = np.random.default_rng(3)
days = pd.bdate_range("2026-01-01", periods=120)
scale = np.where(days.month > 3, 0.02, 0.01)
ret = pd.Series(rng.normal(0, scale), index=days)
ax = ret.plot(alpha=0.4, label="return")
ret.rolling(20).std().plot(ax=ax, label="20-day risk")
ax.legend()
```

Plot



The question

Did risk rise this spring? Daily returns look noisy.

The method

rolling(20).std() measures risk over a moving month of trading days.

Time Series Cheat Sheet

~2 min

Match the task to the tool.

Task	Tool
Parse text to dates	`pd.to_datetime`
Regular dates	`pd.date_range(start, periods, freq)`
Select by date	`ts["2026-02"], ts[a:b]`
Change frequency	`resample, asfreq`
Fill new slots	`ffill, interpolate`
Compare with the past	`shift, diff, pct_change`
Moving statistics	`rolling(n), expanding()`
Time zones	`tz_localize, tz_convert`

Plot the series before and after every resample to catch surprises.